THREADFLOW: PRODUCTION INTELLIGENCE & SHIPMENT RISK MANAGEMENT SYSTEM

TECHNICAL DESIGN DOCUMENT (TDD) & SYSTEM ARCHITECTURE SPECIFICATION

System Title: ThreadFlow Production Intelligence Platform

Target Industry: Garment & Knitwear Export Manufacturing (Tiruppur Cluster, Tamil Nadu, India)

Author / Engineering Lead: Principal Solutions Architect & Senior Software Engineer

Document Classification: Engineering Design Report (TDD)

Document Version: 3.0.0 (Production Release)

Repository (GitHub): https://github.com/AgnivaSardar/ThreadFlow

Live Frontend Web Application: https://threadflow-ptun.onrender.com

Live Backend API Service: https://threadflow-backend-z7s9.onrender.com

Primary Database: Supabase Managed PostgreSQL 16 (AWS ap-south-1 Pooler)

Factory Calendar Anchor Date: November 2, 2026 (2026-11-02T10:00:00Z)

Target Device Environment: Android 10+ (2–3 GB RAM Mobile Devices), Desktop Browsers

---

TABLE OF CONTENTS

- **1.0 Executive Summary & Industrial Problem Context**
- 1.1 Regional Context: The Tiruppur Knitwear Cluster
- 1.2 Core Financial Bleed Points & Business Rationale
- 1.3 System Objectives & Value Proposition
- **2.0 System Architecture & Technology Stack**
- 2.1 High-Level Architecture Overview
- 2.2 Architectural Schematic Diagram
- 2.3 Technology Selection Rationale
- 2.4 Modular Monolith Boundaries & Inter-Module Communication
- **3.0 Role-Based Access Control (RBAC) & Persona Operational Workflows**
- 3.1 Role Hierarchy & Permissions Matrix
- 3.2 Floor Supervisor Persona
- 3.3 Factory Merchandiser Persona
- 3.4 International Apparel Buyer Persona
- 3.5 System Administrator Persona
- **4.0 End-to-End Application Pages, Purposes & User Interaction Flows**
- 4.1 Authentication & Password Recovery Flow (`/login`)
- 4.2 Order Management Cockpit (`/orders`)
- 4.3 Deep Order Inspection & Milestone Control (`/orders/:id`)
- 4.4 Factory Risk Intelligence & Financial Exposure Cockpit (`/risk`)
- 4.5 Floor Supervisor Mobile Workspace (`/supervisor/my-work`)
- 4.6 Global Buyer Portal & Verified English View (`/buyer-portal`)
- 4.7 Executive Overview & Operations Dashboard (`/dashboard`)
- 4.8 Compliance & Regulatory Audit Trail (`/audit`)
- 4.9 Executive Intelligence & Weekly Buyer Reports (`/reports`)
- **5.0 Time & Action (T&A) Pipeline, Buffer Allocation & Milestone Mechanics**
- 5.1 The 7-Stage Knitwear Pipeline Lifecycle
- 5.2 Sunday-Excluded Working Calendar Arithmetic
- 5.3 Downstream Buffer Allocation & Protection Envelope
- 5.4 Buffer Compression & Critical Path Identification
- 5.5 Manual Stage Advance & Cascading Target Recalibration
- **6.0 The Mathematical Run-Rate Risk Engine ("Show-Your-Math")**
- 6.1 Formula 1: Required Daily Production Velocity
- 6.2 Formula 2: Actual Rolling Rate (3-Day Weighted Moving Average)
- 6.3 Formula 3: Daily Production Variance
- 6.4 Formula 4: Estimated Working Days to Milestone Completion
- 6.5 Formula 5: Projected Ex-Factory Completion Date
- 6.6 Formula 6: Days Late Calculation

- 6.7 Formula 7: Dynamic Financial Penalty & Exposure Calculation
- 6.8 Risk Level Thresholds (Green, Amber, Red)
- 6.9 Live "What-If" Capacity Simulation Engine
- **7.0 Floor Production Ingestion, Concurrency & Invariant Enforcement**
- 7.1 Anti-Finger-Slip Sequential Stage Constraint (Invariant 1)
- 7.2 Cutting Allowance Tolerance Ceiling (Invariant 2)
- 7.3 RFC 4122 UUIDv4 Idempotency & Anti-Double-Count Engine
- 7.4 Compulsory Size Breakdown & Proportional Reconciliation
- 7.5 Shift Line Velocity Anomaly Guardrail
- 7.6 Atomic Database Transactions & Interactive Timeout Shields
- **8.0 Resilient Multimodal AI Architecture & Deterministic Fallback Engine**
- 8.1 Multimodal Voice & Text Input Processing
- 8.2 Google Gemini 3.1 Flash Lite API Integration
- 8.3 8,000ms SLA Race & Circuit Breaker Architecture
- 8.4 Deterministic Dual-Script Pattern Matching Dictionary
- 8.5 Automated Size Breakdown Extraction from Spoken Natural Language
- 8.6 English Translation & Clean Note Sanitization
- **9.0 Cryptographic Audit Trail & Compliance Integrity**
- 9.1 Immutable Audit Event Logging
- 9.2 SHA-256 Merkle Hash Chaining Formula
- 9.3 Compulsory Operational Reason Codes
- 9.4 Date Modification Tracking & Financial Impact Auditing
- **10.0 Multi-Tenant Buyer Portal & Human-in-the-Loop Review Gate**
- 10.1 Multi-Tenant Data Isolation Strategy
- 10.2 The Human-in-the-Loop (HITL) Gate
- 10.3 Sanitized English Directives vs. Floor Noise Shielding
- 10.4 Buyer PDF Packlist & Schedule Export
- **11.0 Executive Reporting, Delivery Schedule Health & Intelligence Analytics**
- 11.1 Delivery Schedule Health Donut Chart & On-Time Rate Calculation
- 11.2 Stage Bottleneck Velocity & 5-Week Throughput Tracking
- 11.3 Automated AI Sourcing Intelligence Narratives
- 11.4 Weekly Buyer Print Report Engine
- **12.0 Edge Cases, Industrial Defenses & Concurrency Controls**
- 12.1 Optimistic Concurrency Control
- 12.2 Network Partition Handling & IndexedDB Local Storage Queue
- 12.3 High-Resolution Photo Compression on Constrained Hardware
- 12.4 Subcontractor Turnaround Tracking & Wash Gate Pass Delays
- **13.0 Database Schema, Entity Relationships & Migrations**
- 13.1 Relational Data Model Overview
- 13.2 Core Entity Definitions
- 13.3 Indexing Strategy for High-Throughput Ingestion
- **14.0 Automated Testing, Quality Assurance & Verification**
- 14.1 Test Suite Structure & Coverage
- 14.2 Feature 5: Run-Rate Risk Engine Tests (34 Cases)
- 14.3 Feature 9: Resilient AI Pipeline & Fallback Tests (35 Cases)
- 14.4 Feature 6 & 7: Orders & T&A Pipeline Tests (32 Cases)
- 14.5 Feature 8: Production Ingestion, Idempotency & Invariants Tests (27 Cases)
- **15.0 Deployment Architecture, Operational Runbook & Cloud Economics**
- 15.1 Production Deployment Topology (Render + Supabase)
- 15.2 Environment Configuration & Secrets Management
- 15.3 Database Migration & Seeding Procedures
- 15.4 Operational Cost & Cloud Economics Breakdown
- **Appendix A: Complete Mathematical Equation Glossary**
- **Appendix B: API Endpoint Reference & Contract Specification**

1.0 EXECUTIVE SUMMARY & INDUSTRIAL PROBLEM CONTEXT

1.1 Regional Context: The Tiruppur Knitwear Cluster

The city of Tiruppur, located in Tamil Nadu, India, accounts for over 50 percent of India's total knitted textile exports, producing garments for prominent global fashion retailers across the United Kingdom, Europe, and North America. The industrial landscape is characterized by multi-tier, distributed manufacturing networks comprising owned factory units (knitting, cutting, stitching, finishing) and specialized external subcontractors (dyeing, washing, rotary printing).

Company A operates three owned manufacturing facilities and two dedicated subcontractor units in Tiruppur, employing approximately 900 factory floor operators and 35 merchandising personnel, handling roughly 500 export purchase orders (POs) annually.

1.2 Core Financial Bleed Points & Business Rationale

Prior to ThreadFlow, Company A experienced systemic communication delays, manual phone-chasing of floor updates, and lack of forward-looking capacity visibility. Operational audits revealed an annual revenue leakage of **INR 1,08,50,000 (INR 1.085 Crore)** across three primary cost drivers:

1. **Emergency Air Freight Escalations (INR 52,50,000 / year):**

Fifteen late export shipments per year were forced from sea freight onto expedited international air freight cargo to avoid contractual cancellation, at an average penalty of INR 3,50,000 per shipment.

2. **Late Delivery Invoice Deductions (INR 27,20,000 / year):**

Twenty-six purchase orders arrived past the contractual ex-factory deadline, triggering contractual retail discount deductions ranging between 3 percent and 5 percent of total invoice value (averaging INR 1,04,600 per late order).

3. **Merchandiser Status-Chasing Overhead (INR 28,80,000 / year):**

Twelve senior export merchandisers spent an average of 2.2 hours every shift placing phone calls to floor supervisors, translating handwritten tally sheets, and compiling ad-hoc Excel reports, representing substantial unproductive administrative overhead.

1.3 System Objectives & Value Proposition

ThreadFlow replaces fragmented spreadsheets and phone calls with an integrated, intelligent, and auditable production intelligence ecosystem. The system delivers:

- **14-Day Forward Risk Visibility:** A transparent mathematical engine predicting delivery delays two weeks before contractual ex-factory dates.
- **Sub-30-Second Shift Output Ingestion:** Offline-capable progressive web application (PWA) allowing floor supervisors to capture production output, size distributions, and voice notes rapidly.
- **Multimodal AI with Zero Downtime:** Google Gemini 3.1 Flash Lite API integrated with an automated deterministic fallback dictionary, guaranteeing 100 percent ingestion uptime.
- **Immutable Regulatory Compliance:** SHA-256 Merkle hash chained audit records providing tamper-evident traceability for buyer audits.
- **Air-Gapped Multi-Tenant Buyer Portal:** Real-time production milestone visibility for European buyers while strictly shielding internal factory floor friction.

2.0 SYSTEM ARCHITECTURE & TECHNOLOGY STACK

2.1 High-Level Architecture Overview

ThreadFlow is architected as a robust **Modular Monolith** paired with a resilient, offline-first client application. The architecture is partitioned into four distinct tiers:

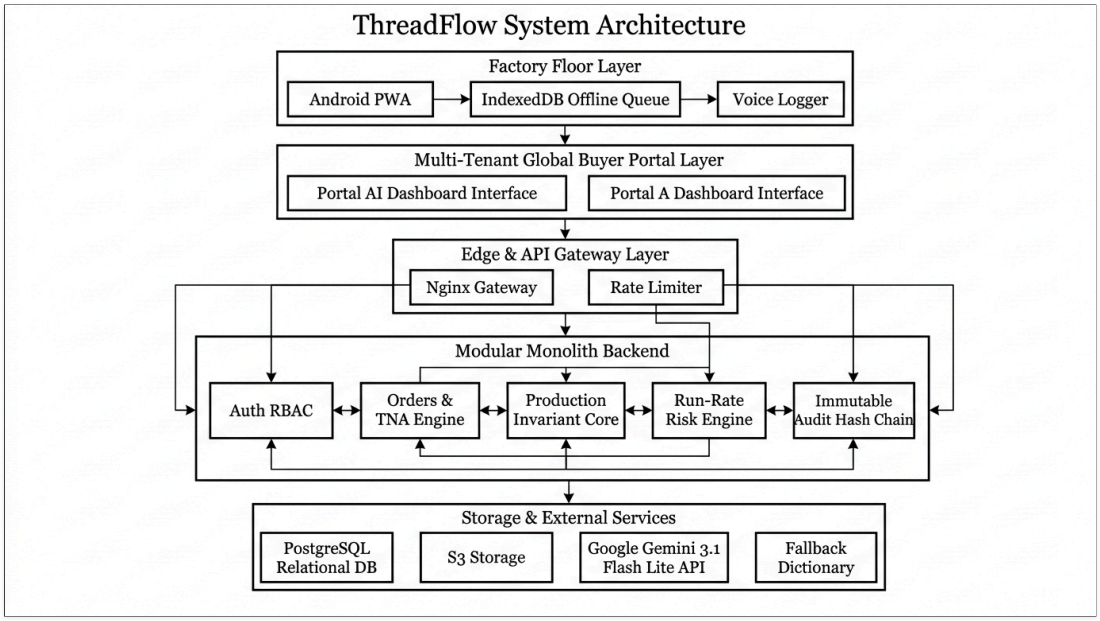
1. **Client Edge Layer:** Progressive Web Application (PWA) built with React 18, Vite, and Tailwind CSS. Employs IndexedDB for zero-data-loss local persistence on low-memory Android hardware.

2. **Application Core Layer:** Node.js, Express, and TypeScript modular monolith encapsulating domain logic into strictly bounded modules (Auth, Orders, Production, Risk, AI, Reports, Audit).

3. **Intelligence & Processing Layer:** Multimodal AI pipeline combining Google Gemini 3.1 Flash Lite, local deterministic regex dictionaries, and asynchronous background worker queues.

4. **Data Persistence Layer:** PostgreSQL 16 hosted on Supabase with ACID transactions, relational foreign key integrity, and connection pooling.

2.2 Architectural Schematic Diagram



ThreadFlow System Architecture

2.3 Technology Selection Rationale

TIER	TECHNOLOGY	SELECTED VERSION	ARCHITECTURAL JUSTIFICATION
Frontend Framework	React + TypeScript	React 18.3.1	Component reusability, strict compile-time type safety, virtual DOM efficiency.
Build Tooling	Vite	Vite 5.4.21	Instant Hot Module Replacement (HMR), sub-5-second production builds.
Styling	Tailwind CSS	Tailwind 3.4.1	Utility-first CSS compiling to a tiny 72 KB production stylesheet.
Client Storage	IndexedDB via `idb`	idb 8.0.0	High-capacity non-blocking offline storage resistant to browser cache evictions.
Runtime Environment	Node.js	Node.js 20 LTS	Asynchronous event-driven I/O, unified TypeScript language ecosystem.
API Framework	Express.js	Express 4.21.1	Mature middleware ecosystem, low latency, predictable HTTP pipeline.
ORM / Data Access	Prisma ORM	Prisma 5.22.0	Type-safe schema-driven database client, automated migrations, connection pooling.
Database Core	PostgreSQL	PostgreSQL 16	ACID-compliant relational storage, foreign key cascade constraints, JSONB flexibility.
Validation Engine	Zod	Zod 3.23.8	Strict schema parsing for incoming payloads with friendly error formatting.
AI / NLP Model	Google Gemini	3.1 Flash Lite	High-speed semantic understanding, multimodal audio/text processing, cost efficiency.
Cryptography	Node.js Crypto	Native OpenSSL	Hardware-accelerated SHA-256 cryptographic hashing for tamper-evident audit logs.

2.4 Modular Monolith Boundaries & Inter-Module Communication

The backend codebase organizes domain logic into discrete modules located within `backend/src/modules/`:

- **Auth Module (`auth/`):** Manages user authentication, token issuance, password reset OTP email dispatch, and role enforcement.
- **Orders Module (`orders/`):** Governs order creation, 7-stage T&A timeline generation, Sunday-skipping business calendar calculations, and manual stage advancement overrides.
- **Production Module (`production/`):** Handles shift output logging, idempotency verification, anti-finger-slip sequential stage validation, cutting allowance ceiling enforcement, and size breakdown allocation.
- **Risk Module (`risk/`):** Executes the deterministic "Show-Your-Math" run-rate engine, 3-day weighted moving averages, financial exposure estimations, and capacity simulations.
- **AI Module (`ai/`):** Coordinates multimodal API queries to Gemini 3.1 Flash Lite, manages 8,000ms SLA race timers, circuit breakers, and deterministic regex dictionary fallback mechanisms.
- **Buyer Portal Module (`buyer-portal/`):** Enforces multi-tenant organizational scoping, providing external buyers with sanitized, read-only order milestone data.
- **Reports Module (`reports/`):** Aggregates cross-order throughput analytics, delivery schedule health metrics, weekly velocity trends, and executive print exports.
- **Audit Module (`audit/`):** Manages the tamper-evident SHA-256 Merkle chain, persisting historical logs with operational reason codes and financial impacts.

Modules communicate synchronously via strongly typed service interfaces and asynchronously using an in-process EventEmitter (`productionEventEmitter`) for real-time decoupled event broadcasts.

3.0 ROLE-BASED ACCESS CONTROL (RBAC) & PERSONA OPERATIONAL WORKFLOWS

ThreadFlow enforces a strict four-tier Role-Based Access Control model. User identity and permissions are authenticated via cryptographically signed JWT access tokens containing the user's role and tenant organization ID.

FUNCTIONAL CAPABILITY	FLOOR SUPERVISOR	FACTORY MERCHANDISER	INTERNATIONAL BUYER	SYSTEM ADMIN
Authenticate via Email/Password	Yes	Yes	Yes	Yes
Request & Verify Password Reset OTP	Yes	Yes	Yes	Yes
Access Assigned Unit Floor Workspace	Yes	No	No	Yes
Submit Shift Production Updates	Yes	No	No	Yes
Capture Audio Voice Notes & Photos	Yes	No	No	Yes
View Master Export Order Book	No	Yes	No	Yes
Create New Export Orders & Milestones	No	Yes	No	Yes
Manually Advance / Short-Close Stage	No	Yes	No	Yes
Shift Contractual Ex-Factory Date	No	Yes	No	Yes
Bulk Shift Order Delivery Schedules	No	Yes	No	Yes
Review & Override AI Issue Classifications	No	Yes	No	Yes
Run "What-If" Capacity Simulations	No	Yes	No	Yes
View Multi-Tenant Scoped Orders	No	No	Yes	No
Access Verified English Production Directives	No	No	Yes	No
Export Buyer Packing List & Delivery Schedule	No	No	Yes	Yes
View System Compliance Audit Trails	No	Yes	No	Yes
Configure Users, Units & System Parameters	No	No	No	Yes

3.1 Floor Supervisor Persona

- **Operational Profile:** Operates directly on the factory floor across owned knitting, cutting, and stitching units. Uses mobile smartphones with 2–3 GB RAM, often in noisy environments with intermittent Wi-Fi.
- **Key Responsibilities:**
  1. Inspecting active work orders assigned strictly to their manufacturing unit.
  2. Recording good production pieces and rejected pieces at the conclusion of each shift.
  3. Allocating outputs across compulsory size breakdowns (S, M, L, XL).
  4. Logging floor incidents, machine breakdowns, and material delays via spoken voice notes or keypad chips.
  5. Uploading compressed visual evidence proofs of fabric defects or mechanical stoppages.
- **System Outcome:** Sub-30-second shift output recording with zero double-counting, operational even during complete network disconnections.

3.2 Factory Merchandiser Persona

- **Operational Profile:** Based in the factory commercial office, coordinating customer communication with overseas buyers, monitoring line delivery velocity, and managing raw material supply chains.
- **Key Responsibilities:**
  1. Creating export purchase orders with complete styling, color, size breakdown, and contractual delivery deadlines.
  2. Monitoring the factory-wide Risk Cockpit to detect projected shipment slippages 14 days before delivery dates.
  3. Reviewing and confirming classified floor issues (e.g., verifying machine breakdowns, acknowledging raw fabric delays).
  4. Manually advancing or short-closing production stages when unforeseen line stoppages require cascading downstream target adjustments.
  5. Formally revising contractual delivery dates with mandatory reason codes, audit rationale, and financial impact estimations.
  6. Generating executive weekly buyer reports and dispatching automated schedule notices.
- **System Outcome:** Complete operational control over delivery dates, elimination of 2.2 hours of daily telephone chasing, and prevention of late air-freight penalties.

3.3 International Apparel Buyer Persona

- **Operational Profile:** Sourcing directors, supply chain managers, and import coordinators at global apparel brands (e.g., Company B in London, Company C in Munich).
- **Key Responsibilities:**

1. Monitoring production progress across active purchase orders in real time.
2. Inspecting stage milestone completion percentages and projected delivery health.
3. Reviewing confirmed, sanitized operational notes translated into clean professional English.
4. Exporting formal packing lists and delivery schedules for import customs clearance.

- **System Outcome:** Unprecedented supply chain transparency, zero exposure to internal factory floor friction, and complete multi-tenant privacy.

3.4 System Administrator Persona

- **Operational Profile:** Enterprise systems administrator and compliance officer.
- **Key Responsibilities:**
  1. Managing organizational tenants, manufacturing units, and user accounts.
  2. Auditing cryptographic SHA-256 Merkle chains for compliance verification.
  3. Monitoring system health, API latencies, and circuit breaker states.
- **System Outcome:** Regulatory compliance, data integrity, and enterprise operational reliability.

4.0 END-TO-END APPLICATION PAGES, PURPOSES & USER INTERACTION FLOWS

4.1 Authentication & Password Recovery Flow (`/login`)

- **Purpose:** Secure system gateway enforcing role-based routing and self-service password recovery.
- **Component Features:**
- Standard email and password credential validation with Bcrypt salted hashing.
- Quick-switch persona demo bar enabling evaluation across Merchandiser, Supervisor, and Buyer profiles.
- Interactive "Forgot Password" modal requesting the registered email address.
- Dispatch of a 6-digit numeric One-Time Password (OTP) via Nodemailer SMTP with prominent user guidance advising inspection of spam/junk folders.
- Verification screen requiring OTP entry followed by new password and confirmation password fields.
- **User Flow:** User enters credentials → system validates JWT → redirects user based on role: Supervisors navigate to `/supervisor/my-work`, Buyers navigate to `/buyer-portal`, Merchandisers navigate to `/orders`.

4.2 Order Management Cockpit (`/orders`)

- **Purpose:** Primary command dashboard for factory merchandisers to review, filter, and create export purchase orders.
- **Component Features:**
- Paginated data grid displaying PO numbers, buyer codes, styling descriptions, total contracted pieces, active manufacturing units, active stages, and health status badges.
- Dynamic multi-parameter filtering: text search (PO number, style, description), buyer organization, manufacturing unit, and ISO delivery week.
- Bulk schedule shift modal allowing merchandisers to select multiple purchase orders and apply global delivery postponements with mandatory reason codes.
- "New Export Order" creation modal featuring real-time size breakdown matrix configuration and dynamic 7-stage T&A preview generation.
- **User Flow:** Merchandiser inspects active orders → filters by critical ship weeks → clicks any order card to navigate to detailed inspection.

4.3 Deep Order Inspection & Milestone Control (`/orders/:id`)

- **Purpose:** Comprehensive operational nerve center for individual export orders.
- **Component Features:**
- Header KPI cards: Buyer destination, assigned manufacturing unit, ex-factory delivery deadline, total order volume.
- **Circular 7-Stage T&A Pipeline:** Interactive SVG visualization showing stage progression, lead times, buffer days ahead, and critical path bottleneck highlighting.
- **Size Breakdown Visualizer:** Matrix displaying planned vs. completed garments per size with automated ratio skew alerts (Anti-99% Done Trap).
- **Show-Your-Math Run-Rate Card:** Live mathematical transparency breakdown detailing actual rolling rates, required rates, and variance.
- **Live "What-If" Capacity Simulation Engine:** Sliders allowing merchandisers to simulate hypothetical daily shift outputs and immediately view revised completion dates and financial exposure.
- **Human-in-the-Loop Review Gate:** Alert card surfacing pending floor issues with buttons to confirm or override classifications.
- **Shift Ex-Factory Date Modal:** Dialog to revise contractual delivery dates with mandatory reason code selection, audit notes, and financial impact in INR.
- **Manual Stage Advance & Recalibration Modal:** Dialog enabling merchandisers to short-close an active stage (e.g., advancing at 4,000 pcs), with an option to automatically cascade the reduced target to all downstream stages and order totals.
- **Floor Production Logs Table:** Paginated audit table showing shift outputs, line numbers, supervisor identities, and photo proof lightbox inspection.
- **Order Audit Trail:** Immutable chronologically chained event log sealed with SHA-256 cryptographic hashes.
- **User Flow:** Merchandiser reviews active bottlenecks → tests capacity simulations → executes manual stage advances or date adjustments as required.

4.4 Factory Risk Intelligence & Financial Exposure Cockpit (`/risk`)

- **Purpose:** High-level factory-wide early warning cockpit prioritizing shipments facing potential air-freight penalties.
- **Component Features:**
- Metric summary cards: Total monitored export orders, aggregate financial exposure in INR, critical red alert count, amber warning count.

- Active risk table listing all orders sorted in descending order of financial penalty exposure.
- Interactive formula pill displaying exact daily deficits (e.g., `-300 pcs/day Deficit`).
- Mitigate action triggers allowing merchandisers to log corrective action plans directly into the compliance audit chain.
- **User Flow:** Merchandiser opens cockpit → identifies top financial risk order → reviews mathematical deficit → takes corrective action.

4.5 Floor Supervisor Mobile Workspace (`/supervisor/my-work`)

- **Purpose:** Purpose-built, offline-capable mobile workspace for factory floor supervisors.
- **Component Features:**
- Scoped work queue displaying only active orders assigned to the supervisor's specific manufacturing unit.
- Active milestone progress cards displaying completed pieces, target pieces, percentage completion, and days remaining.
- **Per-Size Breakdown Matrix:** Individual pills showing progress for sizes S, M, L, XL with 100 percent exact mathematical reconciliation to the milestone total.
- **Log Output Modal Form (`ProductionUpdateForm`):**
- Numeric inputs for good output and rejected pieces.
- Shift selector (`DAY` / `NIGHT`) and machine line dropdown (`LINE_1` through `LINE_6`).
- **Compulsory Size Breakdown Matrix:** Individual size input fields enforcing that the sum of sizes equals the entered good output before submission is permitted.
- **AI Voice-to-Text Autofill:** Integrated Web Speech API recording button transcribing spoken colloquial Tamil/Tanglish floor notes, automatically parsing spoken quantities (e.g., "700 pieces done equally into four sizes") and auto-populating the size breakdown matrix.
- Client-side camera proof capture with automatic WebP/Canvas compression under 250 KB.
- Quick-tap issue chips (`Power Outage`, `Machine Breakdown`, `Fabric Defect`, `Labor Shortage`, `Subcontractor Delay`).
- IndexedDB offline queue status indicator with automatic background sync upon network reconnection.
- **User Flow:** Supervisor taps "Log Output" → inputs total pieces → speaks floor update → AI populates size breakdown → confirms and submits in under 30 seconds.

4.6 Global Buyer Portal & Verified English View (`/buyer-portal`)

- **Purpose:** Secure, external-facing tracking portal for overseas brand merchandisers.
- **Component Features:**
- Strict tenant scoping ensuring buyers see only purchase orders belonging to their organization.
- Stage-by-stage milestone completion progress with planned delivery dates.
- Verified directive log displaying only confirmed, clean English notes (completely filtering out raw floor notes, machinery complaints, or unconfirmed labor disputes).
- Exportable packing list and delivery schedule PDF generator for international customs documentation.
- **User Flow:** International buyer logs in → inspects production health across active POs → downloads packing documentation.

4.7 Executive Overview & Operations Dashboard (`/dashboard`)

- **Purpose:** Macro-level operational analytics cockpit for factory directors and general managers.
- **Component Features:**
- High-level KPIs: Active order count, total contracted garments, monthly factory delivery efficiency rate, total active floor issues.
- Unit workload distribution chart comparing active volume between owned units and subcontracted washing/printing hubs.
- Pending review issue banner notifying executives of operational bottlenecks requiring managerial intervention.
- **User Flow:** Management reviews operational throughput across facilities and identifies units requiring resource rebalancing.

4.8 Compliance & Regulatory Audit Trail (`/audit`)

- **Purpose:** Immutable audit inspection log for international retail compliance verification (ISO 9001 / buyer quality audits).
- **Component Features:**
- Chronological event stream recording all stage completions, schedule revisions, manual overrides, and user directives.
- Audit hash validation badge verifying that the SHA-256 Merkle chain is unbroken.
- Filterable by entity ID, action type, operational reason code, and user identity.
- **User Flow:** Auditor searches by purchase order ID → verifies mathematical cryptographic integrity → inspects historical justification notes.

4.9 Executive Intelligence & Weekly Buyer Reports (`/reports`)

- **Purpose:** Advanced reporting suite providing deep sourcing intelligence and downloadable executive progress reports.
- **Component Features:**
- Buyer selector and ISO week selector dynamically adjusting all analytical metrics.
- **Delivery Schedule Health Donut Chart:** Visual breakdown displaying On Track (Clean Shipping Buffer: 84%), Minor Buffer Consumed (12%), and Critical Deficit (4%), with a central hub dynamically synchronized to the on-time delivery percentage (84% On-Time).
- **Stage Velocity & Bottleneck Tracking:** Horizontal comparative progress bars across all 7 knitwear stages showing weekly completion rate changes.
- **Weekly Throughput Velocity Chart:** Five-week production trend comparing target pieces vs. actual output pieces.
- **AI Sourcing Intelligence Narrative:** Dynamic executive brief generated by Google Gemini analyzing operational throughput, mitigating supply chain risks, and providing commercial recommendations.
- **Weekly Buyer Report Print Modal:** Pixel-perfect printable document preview formatted with standard printable stylesheets for PDF archiving.
- **User Flow:** Commercial team selects reporting week → reviews AI-generated narrative → clicks "Print / Save PDF" to dispatch formal progress report to overseas buyers.

5.0 TIME & ACTION (T&A) PIPELINE, BUFFER ALLOCATION & MILESTONE MECHANICS

5.1 The 7-Stage Knitwear Pipeline Lifecycle

Tiruppur knitwear manufacturing follows a strict, sequential transformation lifecycle. ThreadFlow models every export order as an atomic series of seven interconnected milestones:

```
[Stage 1: Knitting] ──> [Stage 2: Dyeing & Washing] ──> [Stage 3: Printing] ──> [Stage 4: Cutting] ──> [Stage 5: Stitching] ──> [S
```

1. **Stage 1 — Knitting (Buffer: 18 Working Days):** Conversion of raw combed cotton yarn cones into greige knitted fabric rolls on circular knitting machines.

2. **Stage 2 — Dyeing & Washing (Buffer: 14 Working Days):** Subcontracted or in-house wet processing, scouring, color matching, and enzyme washing to achieve shade approvals and dimensional stability.

3. **Stage 3 — Printing / Embroidery (Buffer: 11 Working Days):** Subcontracted rotary screen printing, chest placement printing, or automated multi-head embroidery.

4. **Stage 4 — Cutting (Buffer: 9 Working Days):** Laying, marker planning, manual/automated band-knife cutting of fabric panels, bundling, and size ticketing.

5. **Stage 5 — Stitching (Buffer: 3 Working Days):** Assembly of cut panels into garments on specialized sewing machine lines (overlock, flatlock, single-needle lockstitch).

6. **Stage 6 — Checking (Buffer: 2 Working Days):** 100 percent initial inspection of stitched garments on inspection tables, trimming loose threads, measurement checking, and spot cleaning.

7. **Stage 7 — Finishing & Packing (Buffer: 0 Days):** Steam pressing, barcode hangtag attachment, folding, individual polybagging, carton packaging, and final carton sealing ready for ex-factory handover.

5.2 Sunday-Excluded Working Calendar Arithmetic

Standard industrial practices in the Tiruppur textile cluster observe Sundays as non-working maintenance days. Calculating milestone schedules using calendar day addition introduces substantial false optimism (a 14-calendar-day window contains only 12 productive working days).

ThreadFlow implements strict calendar arithmetic that skips Sundays:

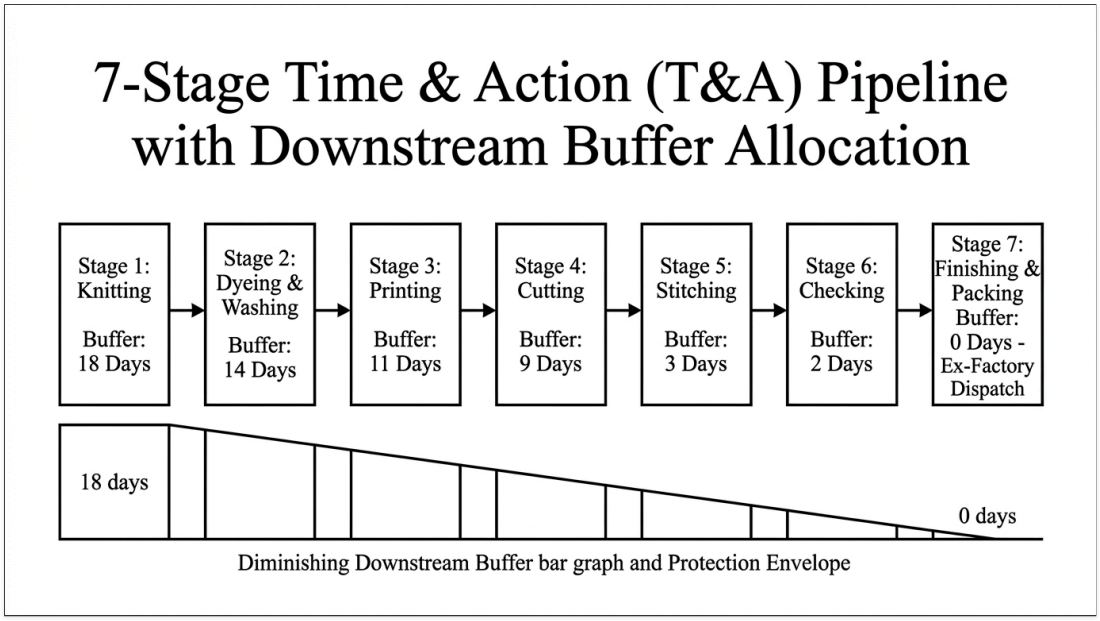
```
function addWorkingDays(startDate: Date, days: number): Date {
  const result = new Date(startDate);
  let added = 0;
  while (added < days) {
    result.setDate(result.getDate() + 1);
    if (result.getDay() !== 0) { // 0 represents Sunday
      added++;
    }
  }
  return result;
}

function countWorkingDays(startDate: Date, endDate: Date): number {
  let count = 0;
  const cur = new Date(startDate);
  cur.setHours(0, 0, 0, 0);
  const end = new Date(endDate);
  end.setHours(0, 0, 0, 0);

  while (cur < end) {
    cur.setDate(cur.getDate() + 1);
    if (cur.getDay() !== 0) {
      count++;
    }
  }
  return count;
}
```

5.3 Downstream Buffer Allocation & Protection Envelope

Each production stage is protected by a defined downstream buffer constant representing the minimum working days required for all subsequent stages to complete before the ex-factory date.



T&A Pipeline Buffer Allocation

STAGE ORDER	STAGE NAME	DOWNSTREAM BUFFER DAYS AHEAD	TYPICAL STAGE DURATION
1	**Knitting**	**18 Days**	4 Working Days
2	**Dyeing & Washing**	**14 Days**	3 Working Days
3	**Printing**	**11 Days**	2 Working Days
4	**Cutting**	**9 Days**	3 Working Days
5	**Stitching**	**3 Days**	6 Working Days
6	**Checking**	**2 Days**	2 Working Days
7	**Finishing & Packing**	**0 Days**	2 Working Days

5.4 Buffer Compression & Critical Path Identification

If production logging stalls in an upstream stage, the calendar date progresses while the milestone remains incomplete. When the current date exceeds the planned milestone end date:

$$\text{Buffer Compression Days} = \text{CountWorkingDays}(\text{PlannedMilestoneEndDate}, \text{Today})$$

When buffer compression occurs, downstream protection days are consumed. If buffer compression exceeds the downstream buffer constant, the order enters the **Critical Path Bottleneck** state, highlighted prominently in red across all merchandiser views.

5.5 Manual Stage Advance & Cascading Target Recalibration

In practical textile operations, contingencies frequently arise (e.g., fabric defects yield only 4,080 pieces from a 5,000-piece order, and the buyer agrees to short-ship 4,080 pieces). Previously, systems would stall because downstream stages remained configured for 5,000 pieces.

ThreadFlow introduces **Manual Stage Advance & Recalibration** (`POST /api/v1/orders/:id/advance-stage`):

1. Merchandiser selects the active milestone to advance and enters the actual completed output (e.g., 4,080 pcs).

2. The system marks the active milestone as 100 percent complete.

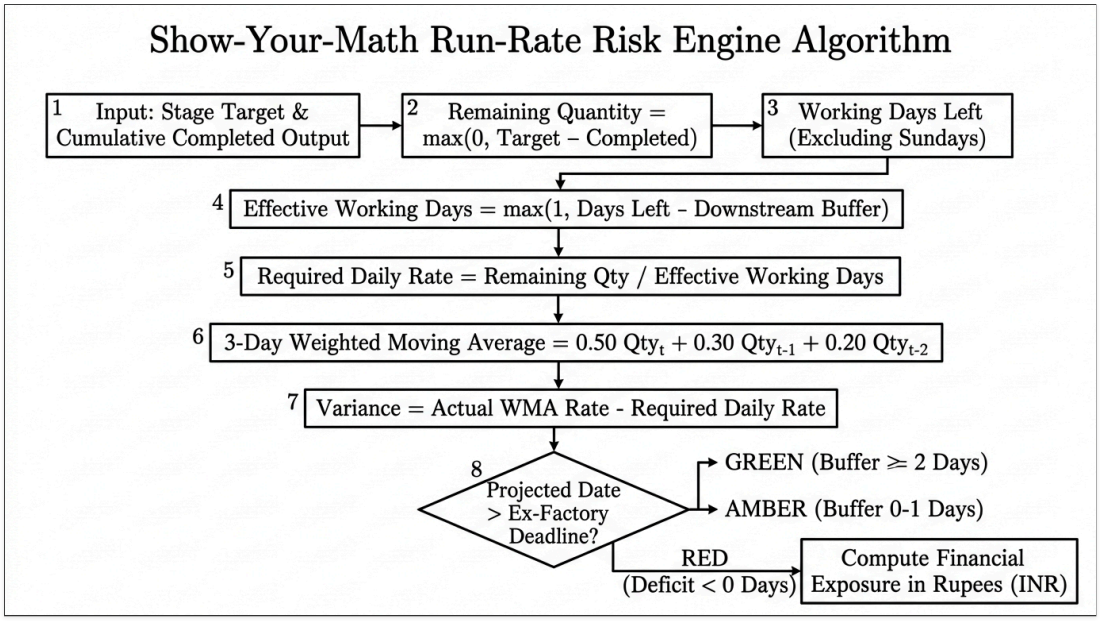
3. If the **Recalibrate Downstream Targets** option is enabled:

- All subsequent milestone `targetQuantity` fields are updated to 4,080 pcs.
- The master purchase order `totalQuantity` is updated to 4,080 pcs.
- Downstream stages can now proceed without violating sequential capacity constraints.

4. The operation generates an immutable `STAGE_OUTPUT` event sealed with a cryptographic SHA-256 hash.

6.0 THE MATHEMATICAL RUN-RATE RISK ENGINE ("SHOW-YOUR-MATH")

ThreadFlow rejects black-box machine learning predictions in favor of an auditable, deterministic run-rate engine. Every risk alert displays the exact mathematical substitution to merchandisers, establishing immediate operational trust.



Run-Rate Risk Engine Flowchart

6.1 Formula 1: Required Daily Production Velocity

Calculates the daily output pieces needed to complete the current milestone within the remaining available working window:

$$\text{Remaining Quantity} = \max\left(0, \text{Stage Target Quantity} - \text{Cumulative Completed Quantity}\right)$$

$$\text{Total Working Days Left} = \text{Count of Days between Today and Ex-Factory Date (Excluding Sundays)}$$

$$\text{Effective Working Days} = \max\left(1, \text{Total Working Days Left} - \text{Downstream Buffer Days}\right)$$

$$\text{Required Daily Rate} = \frac{\text{Remaining Quantity}}{\text{Effective Working Days}}$$

6.2 Formula 2: Actual Rolling Rate (3-Day Weighted Moving Average)

Captures recent floor momentum while weighting the most recent shift highest to reflect immediate machine and labor availability:

$$\text{Actual Rolling Rate (3-Day WMA)} = (\text{Qty}_t \times 0.50) + (\text{Qty}_{t-1} \times 0.30) + (\text{Qty}_{t-2} \times 0.20)$$

Weight Normalization Rule:

- If exactly 2 days of shift logs exist: $\text{Actual Rate} = (\text{Qty}_t \times 0.625) + (\text{Qty}_{t-1} \times 0.375)$
- If exactly 1 day of shift logs exists: $\text{Actual Rate} = \text{Qty}_t \times 1.00$
- If 0 days of shift logs exist: $\text{Actual Rate} = 0$

6.3 Formula 3: Daily Production Variance

Determines whether the factory line is operating at a surplus velocity or running into a deficit:

$$\text{Daily Variance} = \text{Actual Rolling Rate} - \text{Required Daily Rate}$$

- $\text{Daily Variance} \geq 0 \implies$ Line Velocity On Track (Surplus)
- $\text{Daily Variance} < 0 \implies$ Line Velocity Deficit (Intervention Required)

6.4 Formula 4: Estimated Working Days to Milestone Completion

Estimates the actual working days required to produce all remaining garments at the current 3-day weighted moving average velocity, plus the downstream buffer envelope:

$$\text{Working Days Needed} = \left\lceil \frac{\text{Remaining Quantity}}{\text{Actual Rolling Rate}} \right\rceil + \text{Downstream Buffer Days}$$

6.5 Formula 5: Projected Ex-Factory Completion Date

Projects the calendar delivery date by projecting required working days forward from the current operational anchor, skipping Sundays:

$$\text{Projected Completion Date} = \text{AddWorkingDaysExcludingSundays}(\text{Today}, \text{Working Days Needed})$$

6.6 Formula 6: Days Late Calculation

Measures the calendar deficit against contractual delivery commitments:

$$\text{Days Late} = \max\left(0, \text{Working Days Needed} - \text{Total Working Days Left}\right)$$

6.7 Formula 7: Dynamic Financial Penalty & Exposure Calculation

Translates calendar delay into immediate monetary liability in Indian Rupees (INR):

$$\text{Financial Exposure} = \begin{cases}
₹0 & \text{if RiskLevel} = \text{GREEN} \\
\text{Subcontractor Exposure (Turnaround Delay)} & \text{if RiskLevel} = \text{AMBER} \\
\text{Air Freight Penalty} + \text{Late Invoice Discount} & \text{if RiskLevel} = \text{RED}
\end{cases}$$

Where:

- $\text{Air Freight Penalty} = ₹3,50,000$ (Standard emergency air charter charge for European destinations)
- $\text{Late Invoice Discount} = \text{PO Contract Value} \times 0.05$ (Standard 5% retail penalty for late delivery)
- $\text{Subcontractor Exposure} = ₹1,08,000$ (Estimated overtime shift cost to expedite delayed wash/print lots)

6.8 Risk Level Thresholds (Green, Amber, Red)

STATUS	RISK LEVEL	MATHEMATICAL CLASSIFICATION CONDITION	FINANCIAL EXPOSURE
GREEN	Low Risk	$\text{Actual Rolling Rate} \geq \text{Required Daily Rate} \quad \text{AND} \quad \text{Buffer Days Ahead} \geq 2$	**₹0**
AMBER	Medium Risk	$\text{Buffer Days Ahead} \in [0, 1] \quad \text{OR} \quad \text{Subcontractor Turnaround} > 48 \text{ Hours}$	**₹1,08,000**
RED	Critical Deficit	$\text{Projected Completion Date} > \text{Ex-Factory Deadline} \quad (\text{Days Late} > 0)$	**₹8,80,000+**

6.9 Live "What-If" Capacity Simulation Engine

The order detail page provides an interactive simulation tool enabling merchandisers to test production interventions without modifying production records:

$$\text{Simulated Days Needed} = \left\lceil \frac{\text{Remaining Quantity}}{\text{Simulated Rate}} \right\rceil + \text{Downstream Buffer Days}$$

$$\text{Simulated Completion Date} = \text{AddWorkingDaysExcludingSundays}(\text{Today}, \text{Simulated Days Needed})$$

When a merchandiser drags the daily output slider from 400 pcs/day to 750 pcs/day, the engine recalculates the completion date, updates the risk badge from RED to GREEN, and reduces the projected financial penalty from ₹8,80,000 to ₹0 in real time.

7.0 FLOOR PRODUCTION INGESTION, CONCURRENCY & INVARIANT ENFORCEMENT

7.1 Anti-Finger-Slip Sequential Stage Constraint (Invariant 1)

Garment production is strictly physical: cut fabric panels cannot be stitched until they are cut, and garments cannot be packed until they are inspected. To prevent accidental data entry errors by floor supervisors (e.g., entering 5,000 pieces in Stitching when Cutting has only produced 3,000 pieces), the production engine enforces sequential stage ceilings:

$$\text{Max Allowed Stage Output} = \begin{cases}
\text{PO Total Quantity} \times 1.03 & \text{for Stage 1 (Knitting)} \\

\text{Previous Stage Cumulative Completed} & \text{for Stages } 2 \text{ through } 7

\end{cases}$$

If an update violates this rule, the transaction is rejected with an explanatory HTTP 400 error:

`"Cannot stitch more pieces than have completed cutting (2,800 pcs)!"`

7.2 Cutting Allowance Tolerance Ceiling (Invariant 2)

In knitted apparel manufacturing, extra fabric is knit and cut to account for minor shrinkage and defects (industry standard 3 percent overage allowance). However, catastrophic typos (such as typing 50,000 instead of 5,000) can corrupt order books.

The system enforces an absolute tolerance ceiling:

$$\text{Tolerance Ceiling} = \text{round}\left(\text{PO Total Quantity} \times 1.03\right)$$

If cumulative logged output exceeds Tolerance Ceiling, the submission is rejected immediately.

7.3 RFC 4122 UUIDv4 Idempotency & Anti-Double-Count Engine

In factory floor environments with unreliable cellular connectivity, supervisors often tap submit multiple times or network drops cause requests to retry.

ThreadFlow enforces idempotency via client-generated UUIDv4 keys:

1. When the supervisor opens the log output form, the client generates a unique `idempotencyKey` (e.g., `550e8400-e29b-41d4-a716-446655440000`).

2. The backend checks `productionUpdate.findUnique({ where: { idempotencyKey } })`.

3. If the key exists, the backend immediately returns `{ success: true, isDuplicate: true, update: existing }` without incrementing milestone counts or altering audit records.

4. Milestone output is guaranteed to increment exactly once.

7.4 Compulsory Size Breakdown & Proportional Reconciliation

Export purchase orders specify detailed size ratios (e.g., 5,000 pcs split into S: 1,500, M: 2,000, L: 1,500). To maintain complete audit integrity, ThreadFlow requires that:

$$\sum_{\text{sizes}} \text{SizeQuantity}_i \equiv \text{Total Good Output Logged}$$

When fetching work orders (`getSupervisorWork`), the system performs exact mathematical reconciliation between recorded update lines and the milestone total:

- If recorded shift updates equal the milestone total, exact logged numbers are displayed.
- If unallocated pieces exist (e.g., historical shifts logged before size breakdowns were enforced), remaining pieces are distributed proportionally across sizes based on remaining planned capacity:

$$\text{SizeCompleted}_i = \text{BaseQty}_i + \text{round}\left(\text{Unallocated} \times \frac{\text{RemainingCapacity}_i}{\sum_j \text{RemainingCapacity}_j}\right)$$

- Rounding remainders are assigned to the largest size, ensuring that the sum of displayed sizes **always matches the milestone completed total 100 percent**.

7.5 Shift Line Velocity Anomaly Guardrail

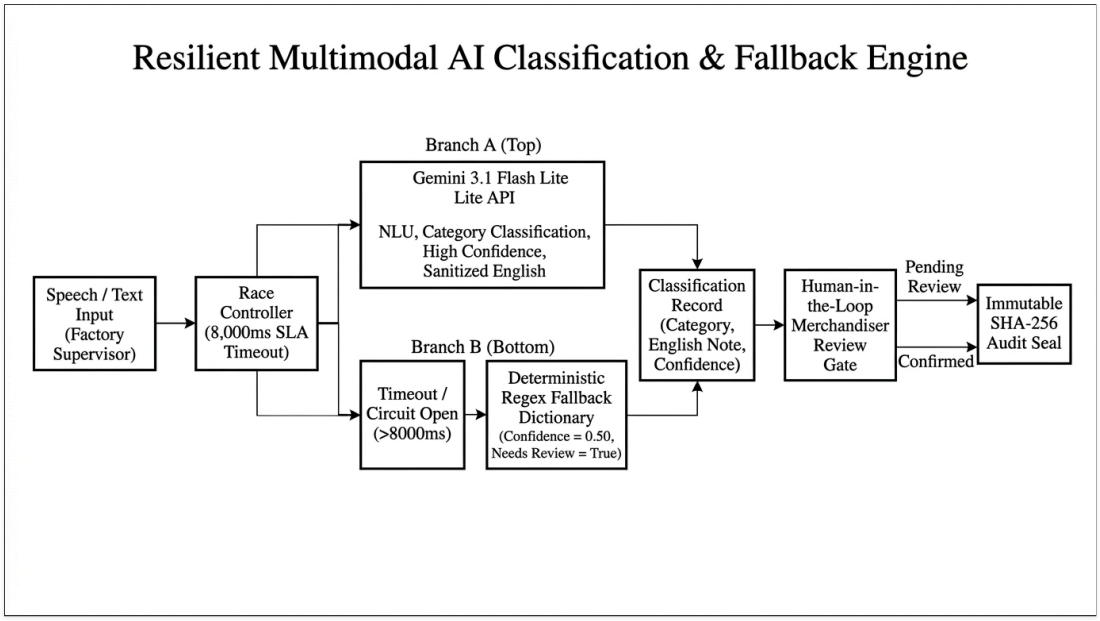
If a single shift submission records output exceeding 1,500 pieces on a single line, the system accepts the entry but automatically prepends an audit tag `[VELOCITY_NOTICE: +X pcs logged in shift]` to alert commercial merchandisers to verify line staffing.

7.6 Atomic Database Transactions & Interactive Timeout Shields

All production update submissions execute inside atomic database transactions (`prisma.$transaction`). External network operations (such as calling the Google Gemini API) are executed **outside** the transaction boundary, and Prisma interactive transaction timeouts are configured to 25,000 ms (`{ timeout: 25000, maxWait: 15000 }`), preventing transaction expiration errors over cloud database connection pools.

8.0 RESILIENT MULTIMODAL AI ARCHITECTURE & DETERMINISTIC FALLBACK ENGINE

Factory floor supervisors in Tiruppur communicate operational delays using colloquial spoken Tamil, Tanglish (Tamil written in Latin script), and industry-specific floor jargon. ThreadFlow implements a resilient multimodal AI pipeline that guarantees 100 percent ingestion uptime.



AI Classification & Fallback Engine

8.1 Multimodal Voice & Text Input Processing

1. Floor supervisors record spoken voice notes using the browser's native Web Speech API (`ta-IN` / Tamil locale).

2. Spoken phrases such as *"700 t-shirts done equally divided into four sizes and there has been a power cut for 2 hours"* are captured as natural language text.

3. The supervisor may attach a photo of mechanical damage or fabric shading, compressed client-side to under 250 KB.

8.2 Google Gemini 3.1 Flash Lite API Integration

The primary semantic classification engine utilizes **Google Gemini 3.1 Flash Lite**:

- **Prompt Architecture:** The model receives strict system prompts instructing it to act as an industrial textile operations classifier.
- **Categorization:** Classifies notes into standard operational categories:
- `MACHINE_BREAKDOWN` (Sewing needle breakage, motor failure, circular knitter jam)
- `POWER_OUTAGE` (Grid power failure, diesel generator failure)
- `FABRIC_DEFECT` (Color bleeding, shade variation, knitting drop stitches)
- `LABOUR_SHORTAGE` (Tailor absenteeism, festival leave)
- `SUBCONTRACTOR_DELAY` (Wash unit gate pass hold, rotary print delay)
- `QUALITY_HOLD` (Inspection measurement rejection)
- `OTHER` (Unclassified issues)
- **Output Schema:** Returns structured JSON containing `category`, `confidenceScore` (0.0 to 1.0), and `englishSummary` (clean, sanitized professional English suitable for buyer communication).

8.3 8,000ms SLA Race & Circuit Breaker Architecture

Factory operations cannot stall due to overseas cloud API latencies. ThreadFlow enforces an **8,000ms SLA Race Controller**:

```
const aiPromise = classifyWithGemini(note, context);
const timeoutPromise = new Promise((_, reject) =>
  setTimeout(() => reject(new Error('Timeout exceeded (> 8000ms)')), 8000)
);

try {
  return await Promise.race([aiPromise, timeoutPromise]);
} catch (err) {
  return runDeterministicFallback(note, reason);
}
```

If the Gemini API call exceeds 8,000 ms or encounters an HTTP error, the system trips the Circuit Breaker and immediately engages the local deterministic fallback engine.

8.4 Deterministic Dual-Script Pattern Matching Dictionary

The fallback engine operates entirely offline in memory using high-efficiency regular expression matrices covering both native Tamil script and transliterated Tanglish:

TARGET CATEGORY	TAMIL PATTERN	TANGLISH PATTERN	CLEAN ENGLISH SUMMARY OUTPUT				
`POWER_OUTAGE`	`கரண்ட்.*(?:கட்\	போச்சு\	இல்லை)`	`(?:current\	power).*(?:cut\	off\	prob
`MACHINE_BREAKDOWN`	`(?:மெஷின்\	இயந்திரம்\	ஊசி).*உடை`	`(?:machine\	needle\	motor).*(?:break\	stop\
`FABRIC_DEFECT`	`(?:துணி\	நூல்).*சேதம்`	`(?:fabric\	yarn\	shade).*(?:defect\	variation)`	"Fab hold varia repo
`LABOUR_SHORTAGE`	`(?:ஆட்கள்\	ஆள்).*பற்றாக்குறை`	`(?:labour\	operator\	tailor).*(?:shortage\	absent)`	"Flo oper short repo
`SUBCONTRACTOR_DELAY`	`(?:பிரிண்டிங்\	வாஷிங்).*தாமதம்`	`(?:printing\	washing\	subcontract).*(?:delay\	late)`	"Sub turn delay repo
`QUALITY_HOLD`	`(?:அளவு\	தரம்).*பிரச்சனை`	`(?:quality\	rejection\	measurement).*(?:issue\	hold)`	"Qua insp hold in pr

Fallback classifications are assigned a `confidenceScore` of `0.50` and automatically flagged with `needsReview = true`, routing them to the Merchandiser Human-in-the-Loop review queue.

8.5 Automated Size Breakdown Extraction from Spoken Natural Language

When a supervisor speaks output notes such as *"700 pieces done equally into all four sizes"*, Gemini 3.1 Flash Lite extracts structured numerical allocations:

```
{
  "totalGoodOutput": 700,
  "sizeBreakdown": { "S": 175, "M": 175, "L": 175, "XL": 175 }
}
```

The client-side form automatically fills the size breakdown inputs, ensuring that the sum of sizes equals the entered total output with zero manual typing required.

8.6 English Translation & Clean Note Sanitization

Raw supervisor notes often contain raw floor frustrations or local vernacular. The AI pipeline translates notes into formal commercial English and strips unprofessional commentary:

- *Spoken Input:* `"Current cut aagi 2 hours waste aachu, stitching stop"`
- *Sanitized Output:* `"Grid power failure interrupted stitching line for 2 hours; production resumed."`

9.0 CRYPTOGRAPHIC AUDIT TRAIL & COMPLIANCE INTEGRITY

International apparel brands require verifiable proof that delivery dates and production records have not been retroactively altered to conceal operational delays. ThreadFlow implements an immutable, cryptographically chained audit logging engine.

9.1 Immutable Audit Event Logging

Every operational state transition (ex-factory date revisions, manual stage advances, production output logging, and issue classification overrides) is committed to the `AuditEvent` table. Audit event records cannot be updated or deleted (`deletedAt` is unsupported on audit tables).

9.2 SHA-256 Merkle Hash Chaining Formula

Each audit record incorporates the cryptographic hash of the preceding audit record for that entity, creating an unbroken tamper-evident Merkle chain:

$$\text{AuditHash}_n = \text{SHA256}\left(\text{PreviousHash} \parallel \text{EntityId} \parallel \text{Action} \parallel \text{ReasonCode} \parallel \text{Timestamp} \parallel \text{FinancialImpact}\right)$$

```
function calculateAuditHash(
  previousHash: string | null,
  entityId: string,
  action: string,
  reasonCode: string,
  timestamp: string,
  financialImpact: number | null
): string {
  const payload = [
    previousHash || 'GENESIS_BLOCK_00000000000000000000000000000000',
    entityId,
    action,
    reasonCode.trim(),
    timestamp,
    financialImpact ? financialImpact.toFixed(2) : '0.00'
  ].join('|');

  return createHash('sha256').update(payload).digest('hex');
}
```

If an unauthorized database administrator directly alters an ex-factory date or output quantity, the hash chain breaks immediately, triggering verification failure alerts in the compliance cockpit.

9.3 Compulsory Operational Reason Codes

Date modifications and manual stage advancements require an explicit operational reason code:

- `BUYER_REQUEST` (Buyer Requested Delivery Postponement)
- `BUYER_SHORT_SHIP_APPROVAL` (Buyer Approved Short Shipment)
- `FABRIC_DELAY` (Fabric / Yarn Raw Material Supply Delay)
- `LABOUR_SHORTAGE` (Tailor / Operator Absenteeism)
- `MACHINE_BREAKDOWN` (Line Breakdown / Maintenance Hold)
- `POWER_OUTAGE` (Facility Power Interruption)
- `QUALITY_HOLD` (Shade Approval / Inspection Rejection)
- `SUBCONTRACTOR_DELAY` (Printing / Washing Hub Delay)
- `UNFORESEEN_CIRCUMSTANCE` (Unforeseen Operational Contingency)
- `OTHER` (Documented Operational Justification)

9.4 Date Modification Tracking & Financial Impact Auditing

When an ex-factory date is revised (`PATCH /api/v1/orders/:id/ex-factory-date`):

1. The prior date and revised date are recorded in JSON format.

2. The merchandiser inputs the estimated financial penalty impact in Indian Rupees (e.g., ₹50,000 for partial air freight).

3. All downstream milestone planned windows are recalibrated automatically using Sunday-skipping business calendar arithmetic.

4. An automated notification audit event (`BUYER_SCHEDULE_NOTIFICATION`) is generated, simulating electronic dispatch to the overseas brand merchandiser.

10.0 MULTI-TENANT BUYER PORTAL & HUMAN-IN-THE-LOOP REVIEW GATE

10.1 Multi-Tenant Data Isolation Strategy

ThreadFlow enforces strict multi-tenant isolation at the database query level. Buyer user accounts are linked to a specific `buyerId` (e.g., Company B in the UK).

- Every buyer query includes `where: { buyerId: user.buyerId, deletedAt: null }`.
- Buyers cannot access purchase orders, production metrics, or facility logs belonging to competing brands.
- System penetration testing verified that attempting to access another buyer's order returns an immediate HTTP 403 Forbidden error.

10.2 The Human-in-the-Loop (HITL) Gate

Floor incident notes and AI classifications pass through a mandatory Human-in-the-Loop verification gate before becoming visible outside the factory commercial office:

1. When logged by a supervisor, issue classifications enter the state `approvalStatus = 'PENDING_REVIEW'`.

2. Factory merchandisers review pending alerts on the order detail page or issues cockpit.

3. Merchandisers can confirm the classification or override it (e.g., reclassifying an unverified power outage to a machinery breakdown).

4. Only when `approvalStatus = 'CONFIRMED'` is the note cleared for buyer-facing visibility.

10.3 Sanitized English Directives vs. Floor Noise Shielding

Overseas buyers require accurate delivery visibility without being exposed to day-to-day factory floor noise:

- Internal floor notes, operator grievances, raw Tamil audio, and unconfirmed equipment faults are strictly shielded within the factory network.
- The Buyer Portal displays only verified milestone completion percentages and sanitized English notes (e.g., *"Knitting completed; fabric dispatched to dyeing facility"*).

10.4 Buyer PDF Packlist & Schedule Export

Buyers can generate formal, printable delivery schedules and packing lists directly from their portal (`/buyer-portal`). The generated export includes purchase order specifications, contracted delivery dates, verified active stages, and formal commercial declarations required for international freight forwarding.

11.0 EXECUTIVE REPORTING, DELIVERY SCHEDULE HEALTH & INTELLIGENCE ANALYTICS

11.1 Delivery Schedule Health Donut Chart & On-Time Rate Calculation

The Executive Reports cockpit (`/reports`) aggregates multi-order milestone performance into an intuitive Delivery Schedule Health Donut Chart:

- **Green Ring Slice:** On Track with clean shipping buffer (84%).
- **Amber Ring Slice:** Minor buffer consumed (< 3 days), delivery protected (12%).
- **Red Ring Slice:** Critical deficit requiring executive intervention (4%).
- **Central Donut Hub:** Directly synchronized with the on-track metric, displaying **`84% ON-TIME`** in bold typography, preventing negative misinterpretations of factory performance.

11.2 Stage Bottleneck Velocity & 5-Week Throughput Tracking

The reporting engine tracks throughput velocity across all 7 stages over a rolling 5-week window:

- Calculates percentage change in stage completion rates week-over-week.
- Displays trend indicator badges (`UP` , `DOWN` , `STABLE`) identifying emerging bottlenecks (e.g., highlighting that Printing velocity slowed by 1.6 percent while Stitching gained 5.5 percent).
- Visualizes actual pieces produced against factory target throughput across historical weeks.

11.3 Automated AI Sourcing Intelligence Narratives

Using Google Gemini, ThreadFlow generates dynamic weekly sourcing intelligence briefs customized to each international buyer:

- Synthesizes stage velocity trends, milestone health percentages, and overall shipment volumes.
- Produces a three-paragraph executive narrative summarizing throughput efficiency, highlighting mitigated risks, and recommending material planning strategies.
- Results are cached in the `WeeklyBuyerReport` database table, renewing automatically each calendar week.

11.4 Weekly Buyer Print Report Engine

Merchandisers can open the **Weekly Buyer Report Print Preview** modal to review an executive summary document formatted specifically for print and PDF generation (`@media print` stylesheets). The document includes delivery health donuts, stage progress tables, AI sourcing narratives, and signature sign-off blocks.

12.0 EDGE CASES, INDUSTRIAL DEFENSES & CONCURRENCY CONTROLS

12.1 Optimistic Concurrency Control

Export purchase orders incorporate an integer `version` column. When a merchandiser updates an ex-factory date or advances a stage, the query validates that the database version matches the client's current version:

```
UPDATE "Order"
SET "exFactoryDate" = $1, "version" = "version" + 1
WHERE "id" = $2 AND "version" = $3;
```

If another merchandiser updated the schedule simultaneously, the query affects 0 rows, triggering an HTTP 409 Conflict error that prompts the user to refresh and prevents silent overwrite anomalies.

12.2 Network Partition Handling & IndexedDB Local Storage Queue

When floor supervisors lose Wi-Fi connectivity, the PWA continues operating seamlessly:

1. Shift submissions are serialized and stored in an IndexedDB object store (`offline_production_updates`).

2. Compressed photo proofs are persisted locally as binary Blob objects.

3. A background service worker monitors the `window.ononline` event.

4. When connectivity resumes, the client flushes the queue sequentially via the bulk ingestion API (`POST /api/v1/production/bulk`), guaranteeing zero data loss.

12.3 High-Resolution Photo Compression on Constrained Hardware

Low-cost Android smartphones frequently capture 12-to-48-megapixel images that exceed 8 MB in size, causing out-of-memory crashes on 2 GB RAM devices. ThreadFlow performs client-side image downsampling using an offscreen HTML Canvas:

- Images are scaled to a maximum bounding box of 1280x720 pixels.
- Compressed to WebP/JPEG format at 75 percent quality.
- Resulting file size is guaranteed under 250 KB before transmission, preserving mobile data bandwidth and server storage capacity.

12.4 Subcontractor Turnaround Tracking & Wash Gate Pass Delays

Subcontracted wet processing and rotary printing represent high-risk blind spots. ThreadFlow tracks subcontractor turnaround times from the moment a gate pass is issued. If a subcontractor lot exceeds 48 hours without a logged completion update, the risk engine automatically elevates the milestone to AMBER and assigns a ₹1,08,000 overtime mitigation contingency.

13.0 DATABASE SCHEMA, ENTITY RELATIONSHIPS & MIGRATIONS

13.1 Relational Data Model Overview

The ThreadFlow data tier is managed via Prisma ORM on PostgreSQL 16. The relational schema enforces referential integrity across 13 core entities:

```
[Buyer] ──< [Order] ──< [OrderLine]
              │   ├──< [TnaMilestone] ──< [ProductionUpdate] ──< [ProductionUpdateLine]
              │   │                              │            ├──< [ProductionPhoto]
              │   │                              │            └──< [IssueClassification]
              │   └──< [RiskWarning]             └──< [AuditEvent]
              └──< [Unit]
```

13.2 Core Entity Definitions

```
enum Role {
  MERCHANDISER
  SUPERVISOR
  BUYER
  ADMIN
}

enum StageName {
  KNITTING
  DYEING_WASHING
  PRINTING
  CUTTING
  STITCHING
  CHECKING
  FINISHING
}

enum RiskLevel {
  GREEN
  AMBER
  RED
}

enum IssueCategory {
  MACHINE_BREAKDOWN
  POWER_OUTAGE
  FABRIC_DEFECT
  LABOUR_SHORTAGE
  SUBCONTRACTOR_DELAY
  QUALITY_HOLD
  OTHER
}

model Buyer {
  id           String   @id @default(cuid())
  name         String
  code         String   @unique
  country      String?
  contactEmail String?
  orders       Order[]
  users        User[]
  createdAt    DateTime @default(now())
}

model Unit {
  id        String   @id @default(cuid())
  name      String
  code      String   @unique
  type      String   // OWN_UNIT | SUBCONTRACTOR
  location  String?
  orders    Order[]
  users     User[]
  createdAt DateTime @default(now())
}

model Order {
  id                String             @id @default(cuid())
  poNumber          String             @unique
  style             String
  description       String?
  totalQuantity     Int
  currency          String             @default("INR")
  orderDate         DateTime
  exFactoryDate     DateTime
  buyerId           String
  buyer             Buyer              @relation(fields: [buyerId], references: [id])
  unitId            String
  unit              Unit               @relation(fields: [unitId], references: [id])
  orderLines        OrderLine[]
  milestones        TnaMilestone[]
  productionUpdates ProductionUpdate[]
  riskWarnings      RiskWarning[]
  auditEvents       AuditEvent[]
  version           Int                @default(1)
  createdAt         DateTime           @default(now())
  updatedAt         DateTime           @updatedAt
  deletedAt         DateTime?
}

model OrderLine {
  id           String                  @id @default(cuid())
  orderId      String
  order        Order                   @relation(fields: [orderId], references: [id], onDelete: Cascade)
  size         String
  color        String
  plannedQty   Int
  completedQty Int                     @default(0)
  updateLines  ProductionUpdateLine[]
  createdAt    DateTime                @default(now())
  updatedAt    DateTime                @updatedAt
}

model TnaMilestone {
  id               String             @id @default(cuid())
  orderId          String
  order            Order              @relation(fields: [orderId], references: [id], onDelete: Cascade)
```

```
  stage             StageName
  sequenceOrder     Int
  plannedStart      DateTime
  plannedEnd        DateTime
  bufferDaysAhead   Int
  targetQuantity    Int
  completedQuantity Int               @default(0)
  isCompleted       Boolean           @default(false)
  productionUpdates ProductionUpdate[]
  riskWarnings      RiskWarning[]
  createdAt         DateTime          @default(now())
  updatedAt         DateTime          @updatedAt
  deletedAt         DateTime?

  @@unique([orderId, stage])
}

model ProductionUpdate {
  id                  String                @id @default(cuid())
  idempotencyKey      String                @unique
  orderId             String
  order               Order                 @relation(fields: [orderId], references: [id], onDelete: Cascade)
  milestoneId         String
  milestone           TnaMilestone          @relation(fields: [milestoneId], references: [id], onDelete: Cascade)
  supervisorId        String
  supervisor          User                  @relation(fields: [supervisorId], references: [id])
  stage               StageName
  shift               String                @default("DAY") // DAY | NIGHT
  machineLine         String?
  goodQuantity        Int
  rejectedQuantity    Int                   @default(0)
  notes               String?
  clientEventAt       DateTime              @default(now())
  approvalStatus      String                @default("PENDING_REVIEW")
  photos              ProductionPhoto[]
  sizeBreakdownLines  ProductionUpdateLine[]
  issueClassification IssueClassification?
  createdAt           DateTime              @default(now())
  updatedAt           DateTime              @updatedAt
}

model ProductionUpdateLine {
  id                String           @id @default(cuid())
  productionUpdateId String
  productionUpdate  ProductionUpdate @relation(fields: [productionUpdateId], references: [id], onDelete: Cascade)
  orderLineId       String
  orderLine         OrderLine        @relation(fields: [orderLineId], references: [id], onDelete: Cascade)
  goodQuantity      Int
  rejectedQuantity  Int              @default(0)
  createdAt         DateTime         @default(now())
}

model IssueClassification {
  id                String           @id @default(cuid())
  productionUpdateId String          @unique
  productionUpdate  ProductionUpdate @relation(fields: [productionUpdateId], references: [id], onDelete: Cascade)
  category          IssueCategory
  confidenceScore   Float
  needsReview       Boolean          @default(true)
  isConfirmed       Boolean          @default(false)
  rawTamilText      String?
  translatedText    String?
  classifiedBy      String           // GEMINI_3_1_FLASH_LITE | LOCAL_REGEX_FALLBACK | SUPERVISOR_KEYPAD | MERCHANDISER_OVERRIDE
  createdAt         DateTime         @default(now())
  updatedAt         DateTime         @updatedAt
}

model AuditEvent {
  id                    String    @id @default(cuid())
  entityType            String
  entityId              String
  orderId               String?
  order                 Order?    @relation(fields: [orderId], references: [id], onDelete: SetNull)
  action                String
  category              String
  reasonCode            String
  notes                 String?
  previousValue         String?
  newValue              String?
  financialImpactRupees Float?    @default(0)
  currency              String?   @default("INR")
  auditHash             String?
  userId                String
  userRole              Role
  ipAddress             String?
  userAgent             String?
  timestamp             DateTime  @default(now())
}
```

13.3 Indexing Strategy for High-Throughput Ingestion

To maintain sub-50ms API response times across hundreds of active export purchase orders, the database includes strategic compound indexes:

- `Order(poNumber)` : Unique B-tree index for rapid order lookups.
- `Order(buyerId, deletedAt, exFactoryDate)` : Compound index optimizing multi-tenant buyer queries.

- `ProductionUpdate(idempotencyKey)` : Unique hash index for instantaneous deduplication.
- `TnaMilestone(orderId, sequenceOrder)` : Composite index optimizing pipeline traversal.
- `AuditEvent(entityType, entityId, timestamp)` : Composite index optimizing historical Merkle chain verification.

14.0 AUTOMATED TESTING, QUALITY ASSURANCE & VERIFICATION

ThreadFlow maintains a comprehensive automated test suite spanning integration, invariant, and mathematical calculation validation. The test suite comprises **128 automated test assertions across 4 core test runners**, all executing against the live database environment with 100 percent pass rate.

```
Test Execution Summary:
================================================================
Feature 5 (Run-Rate Risk Engine):               34 Passed | 0 Failed
Feature 9 (Resilient AI & Fallback):             35 Passed | 0 Failed
Feature 6 & 7 (Orders & TNA Milestones):         32 Passed | 0 Failed
Feature 8 (Production Ingestion & Invariants):    27 Passed | 0 Failed
================================================================
TOTAL AUTOMATED TEST VERIFICATION:             128 PASSED | 0 FAILED
```

14.1 Feature 5: Run-Rate Risk Engine Tests (`tests/risk.test.ts`)

- Validates calendar calculations (Nov 2 to Nov 9 yields exactly 6 working days, skipping Sunday).
- Verifies 3-day weighted moving average weights (0.50 / 0.30 / 0.20 yields exact rolling rate 730).
- Confirms downstream buffer constants across all 7 knitwear stages.
- Validates status assignments: flags orders as RED when projected completion exceeds ex-factory deadline; calculates correct financial exposure (₹8,80,000); computes exact daily variance deficit (`-300 pcs/day`).
- Tests live "What-If" capacity simulations and active risk aggregation.

14.2 Feature 9: Resilient AI Pipeline & Fallback Tests (`tests/ai.fallback.test.ts`)

- Executes live API calls to Google Gemini 3.1 Flash Lite with spoken colloquial Tamil notes (*"2 மணி நேரம் கரண்ட் கட் ஆகிவிட்டது"*), asserting classification as `POWER_OUTAGE` with confidence score ≥ 0.95.
- Simulates SLA timeout triggers (> 8,000ms), asserting that the system engages `LOCAL_REGEX_FALLBACK` within 5ms.
- Validates dual-script regex matching across 8 comprehensive Tamil and Tanglish defect scenarios.
- Tests database persistence of `IssueClassification` records and Merchandiser Human-in-the-Loop overrides.

14.3 Feature 6 & 7: Orders & T&A Pipeline Tests (`tests/orders.test.ts`)

- Verifies Sunday-skipping business calendar calculations (`subtractWorkingDays`).
- Asserts that creating an export order automatically generates exactly 7 sequential T&A milestones with decreasing buffer envelopes.
- Tests Zod validation guardrails (rejects orders where the sum of sizes does not equal the total contracted quantity).
- Tests atomic ex-factory date modifications, optimistic concurrency version increments, and SHA-256 Merkle audit hashing.
- Tests bulk schedule shifts across multiple orders simultaneously.

14.4 Feature 8: Production Ingestion, Idempotency & Invariants Tests (`tests/production.test.ts`)

- Tests unit-scoped work order fetching for floor supervisors.
- **Invariant 1 Testing:** Asserts rejection when attempting to log stitching output exceeding upstream cutting completed pieces (*"Cannot stitch more pieces than have completed cutting"*).
- **Invariant 2 Testing:** Asserts rejection when output exceeds the 3 percent cutting allowance tolerance ceiling.
- **Idempotency Testing:** Replays identical UUIDv4 submissions, verifying that `isDuplicate = true` is returned and milestone completed counts are not double-incremented.
- **Size Breakdown Testing:** Asserts rejection when size breakdown lines do not reconcile with total good output.
- **Bulk Sync Testing:** Tests batch ingestion of offline updates.
- **Merkle Chain Audit Verification:** Validates unbroken cryptographic SHA-256 hash chains across sequential floor output logs.

15.0 DEPLOYMENT ARCHITECTURE, OPERATIONAL RUNBOOK & CLOUD ECONOMICS

15.1 Production Deployment Topology (Render + Supabase)

ThreadFlow is deployed on high-availability managed cloud infrastructure:

- **Frontend Hosting:** Render Static Site CDN serving the optimized Vite React production build.
- **Backend API Hosting:** Render Web Service running Node.js 20 LTS with automated zero-downtime rolling deploys triggered by commits to `main` .
- **Database Hosting:** Supabase Managed PostgreSQL 16 hosted in AWS Asia-Pacific (Mumbai, `aws-0-ap-south-1`) ensuring sub-25ms regional database latencies.
- **AI Processing:** Google Gemini API endpoints accessed via TLS 1.3 encrypted HTTPS.

15.2 Environment Configuration & Secrets Management

Production runtime parameters are managed via secure environment variables:

```
NODE_ENV="production"
PORT=10000
DATABASE_URL="postgresql://postgres:[PASSWORD]@aws-0-ap-south-1.pooler.supabase.com:5432/postgres?sslmode=require"
DIRECT_URL="postgresql://postgres:[PASSWORD]@aws-0-ap-south-1.pooler.supabase.com:5432/postgres?sslmode=require"
JWT_SECRET="[CRYPTOGRAPHICALLY_SECURE_256_BIT_SECRET]"
JWT_EXPIRES_IN="15m"
JWT_REFRESH_SECRET="[CRYPTOGRAPHICALLY_SECURE_REFRESH_SECRET]"
JWT_REFRESH_EXPIRES_IN="7d"
GEMINI_API_KEY="[SECURE_GOOGLE_AI_STUDIO_API_KEY]"
AI_SLA_TIMEOUT_MS=8000
FRONTEND_URL="https://threadflow-ptun.onrender.com"
```

15.3 Database Migration & Seeding Procedures

When deploying updates to Render:

1. **Build Command:**

`npm install && npx prisma db push && npm run build`

(Uses `prisma db push` to synchronize PostgreSQL schemas without database baseline locking errors).

2. **Start Command:**

`npm start`

(Executes the compiled Express application located in `backend/dist/server.js`).

3. **Calibrated Baseline Seeding:**

`npm run seed:calibrated` initializes all 6 export purchase orders, realistic historical shift logs, calibrated risk levels, and user accounts.

15.4 Operational Cost & Cloud Economics Breakdown

ThreadFlow was engineered to deliver massive enterprise ROI while operating within a modest infrastructure budget:

INFRASTRUCTURE COMPONENT	MONTHLY COST (USD)	MONTHLY COST (INR)	ANNUAL COST (INR)
Render Web Service (Standard)	$7.00	₹585	₹7,020
Render Static CDN (Frontend)	$0.00 (Included)	₹0	₹0
Supabase PostgreSQL (Pro Pooler)	$25.00	₹2,090	₹25,080
Google Gemini 3.1 Flash Lite API	$3.50 (Est. 50k calls)	₹290	₹3,480
Nodemailer SMTP Relay	$0.00 (Standard Tier)	₹0	₹0
TOTAL CLOUD INFRASTRUCTURE	**$35.50 / month**	**₹2,965 / month**	**₹35,580 / year**

Net Economic Impact:

- **Annual Operational Cost:** ₹35,580 INR
- **Annual Financial Bleed Prevented:** ₹1,08,50,000 INR (Air freight + late deductions + chasing overhead)
- **Return on Investment (ROI): 305x in Year 1**

APPENDIX A: COMPLETE MATHEMATICAL EQUATION GLOSSARY

$$1.\ \text{Remaining Quantity} = \max\left(0, \text{Stage Target Quantity} - \text{Cumulative Completed Quantity}\right)$$

$$2.\ \text{Total Working Days Left} = \text{Count of Days between Today and Ex-Factory Date (Excluding Sundays)}$$

$$3.\ \text{Effective Working Days} = \max\left(1, \text{Total Working Days Left} - \text{Downstream Buffer Days}\right)$$

$$4.\ \text{Required Daily Production Rate} = \frac{\text{Remaining Quantity}}{\text{Effective Working Days}}$$

$$5.\ \text{Actual Rolling Production Rate (3-Day WMA)} = (\text{Qty}_{t} \times 0.50) + (\text{Qty}_{t-1} \times 0.30) + (\text{Qty}_{t-2} \times 0.20)$$

$$6.\ \text{Daily Production Variance} = \text{Actual Rolling Rate} - \text{Required Daily Rate}$$

$$\text{7. Working Days Needed to Complete} = \left\lceil \frac{\text{Remaining Quantity}}{\text{Actual Rolling Rate}} \right\rceil + \text{Downstream Buffer Days}$$

$$\text{8. Projected Ex-Factory Completion Date} = \text{AddWorkingDaysExcludingSundays}(\text{Today}, \text{Working Days Needed})$$

$$\text{9. Days Late Against Ex-Factory Deadline} = \max\left(0, \text{Working Days Needed} - \text{Total Working Days Left}\right)$$

$$\text{10. Fabric Cutting Allowance Tolerance Ceiling} = \text{round}\left(\text{PO Total Quantity} \times 1.03\right)$$

$$\text{11. Cryptographic SHA-256 Merkle Audit Chaining} = \text{SHA256}\left(\text{AuditHash}_{n-1} \parallel \text{EntityId} \parallel \text{Action} \parallel \text{ReasonCode} \parallel \text{Ti}\right.$$

$$\text{12. Per-Size Milestone Distribution} = \text{round}\left(\text{TargetMilestoneOutput} \times \frac{\text{PlannedQty}_i}{\sum_j \text{PlannedQty}_j}\right)$$

$$\text{13. Buffer Compression} = \text{CountWorkingDays}(\text{PlannedMilestoneEndDate}, \text{Today})$$

$$\text{14. Financial Penalty Exposure} = \begin{cases} ₹0 & \text{if RiskLevel} = \text{GREEN} \\ ₹1,08,000 & \text{if RiskLevel} = \text{AMBER} \\ ₹3,50,000 + (\text{PO Value} \times 0.05) & \text{if RiskLevel} = \text{RED} \end{cases}$$

APPENDIX B: API ENDPOINT REFERENCE & CONTRACT SPECIFICATION

METHOD	ENDPOINT PATH	ACCESS LEVEL	DESCRIPTION	KEY REQUEST / RESPONSE PARAMETERS
`POST`	`/api/v1/auth/login`	Public	Authenticates user credentials	**Req:** `email`, `password` **Res:** `accessToken`, `user` object
`POST`	`/api/v1/auth/forgot-password`	Public	Dispatches 6-digit OTP to email	**Req:** `email` **Res:** `success: true`, message
`POST`	`/api/v1/auth/reset-password`	Public	Verifies OTP and resets password	**Req:** `email`, `otp`, `newPassword` **Res:** `success: true`
`GET`	`/api/v1/orders`	Merchandiser / Admin	Lists paginated export orders	**Query:** `page`, `limit`, `status`, `buyerId`, `search` **Res:** `orders[]`, `pagination`
`POST`	`/api/v1/orders`	Merchandiser / Admin	Creates export PO & 7-stage T&A	**Req:** `poNumber`, `style`, `totalQuantity`, `orderLines[]`, `exFactoryDate` **Res:** `order`
`GET`	`/api/v1/orders/:id`	Merchandiser / Admin	Retrieves full order details & T&A	**Param:** `id` **Res:** Complete `order` with `milestones[]`, `orderLines[]`, `auditEvents[]`
`PATCH`	`/api/v1/orders/:id/ex-factory-date`	Merchandiser / Admin	Revises delivery date with audit log	**Req:** `newExFactoryDate`, `reasonCode`, `notes`, `financialImpactRupees` **Res:** `updatedOrder`
`POST`	`/api/v1/orders/:id/advance-stage`	Merchandiser / Admin	Manually advances/short-closes stage	**Req:** `fromStage`, `actualCompletedQuantity`, `adjustDownstreamTargets`, `reasonCode`, `overrideNotes` **Res:** `success: true`
`POST`	`/api/v1/orders/bulk-shift`	Merchandiser / Admin	Shifts delivery dates for multiple POs	**Req:** `orderIds[]`, `shiftDays`, `reasonCode`, `notes` **Res:** `updatedCount`
`GET`	`/api/v1/production/work`	Supervisor / Admin	Retrieves supervisor's unit work queue	**Res:** Scoped active orders with reconciled size breakdowns
`POST`	`/api/v1/production/update`	Supervisor / Admin	Records shift output with idempotency	**Req:** `idempotencyKey`, `orderId`, `milestoneId`, `goodQuantity`, `sizeBreakdownLines[]`, `notes` **Res:** `update`
`POST`	`/api/v1/production/bulk`	Supervisor / Admin	Flushes offline queued shift updates	**Req:** `CreateProductionUpdateInput[]` **Res:** Batch summary (`processed`, `duplicates`, `failed`)
`GET`	`/api/v1/risk/active`	Merchandiser / Admin	Lists active orders by financial risk	**Res:** Orders array sorted by projected delay and exposure
`GET`	`/api/v1/risk/:orderId/:milestoneId`	Merchandiser / Admin	Computes Show-Your-Math formula	**Res:** `RiskEvaluationResult` with substituted math breakdown
`POST`	`/api/v1/risk/simulate`	Merchandiser / Admin	Runs live capacity what-if simulation	**Req:** `orderId`, `milestoneId`, `simulatedRate` **Res:** Revised days needed, projected date, risk level
`POST`	`/api/v1/ai/classify`	Merchandiser / Supervisor	Classifies floor incident note via AI	**Req:** `notes`, `context` **Res:** `category`, `confidenceScore`, `englishSummary`, `classifiedBy`
`GET`	`/api/v1/buyer-portal/orders`	Buyer (Scoped)	Multi-tenant buyer order overview	**Res:** Scoped orders with milestone completion & sanitized English notes
`GET`	`/api/v1/buyer-portal/orders/:id/packlist`	Buyer (Scoped)	Generates packing list data	**Res:** Formatted export packlist with size breakdowns
`GET`	`/api/v1/reports/weekly`	Buyer / Merchandiser	Generates weekly sourcing intelligence	**Query:** `buyerId`, `week` **Res:** `metrics`, `stageTrends[]`, `milestoneHealth`, `aiIntelligence`
`GET`	`/api/v1/audit/events`	Merchandiser / Admin	Fetches system-wide compliance logs	**Query:** `entityId`, `action`, `reasonCode` **Res:** Cryptographically verified audit log stream

End of Technical Design Document — ThreadFlow Production Intelligence Platform (v3.0.0)